

ML EDA and Linear Regression

CODE WITH OUTPUT (JUPYTER NOTEBOOK):

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.preprocessing import StandardScaler
```

```
df = pd.read_csv("House Price Prediction Dataset.csv")
```

```
df.head()
```

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage	Price
0	1	1360	5	4	3	1970	Downtown	Excellent	No	149919
1	2	4272	5	4	3	1958	Downtown	Excellent	No	424998
2	3	3592	2	2	3	1938	Downtown	Good	No	266746
3	4	966	4	2	2	1902	Suburban	Fair	Yes	244020
4	5	4926	1	4	2	1975	Downtown	Fair	Yes	636056

```
df.shape
```

```
df.info()
```

```
df.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype  
---  -
0    Id              2000 non-null   int64  
1    Area            2000 non-null   int64  
2    Bedrooms        2000 non-null   int64  
3    Bathrooms       2000 non-null   int64  
4    Floors          2000 non-null   int64  
5    YearBuilt       2000 non-null   int64  
6    Location        2000 non-null   object  
7    Condition       2000 non-null   object  
8    Garage          2000 non-null   object  
9    Price           2000 non-null   int64  
dtypes: int64(7), object(3)
memory usage: 156.4+ KB
Index(['Id', 'Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt',
      'Location', 'Condition', 'Garage', 'Price'],
      dtype='object')
```

```
X = df.drop("Price", axis=1)
```

```
y = df["Price"]
```

```
df.isnull().sum()
```

```
df = df.drop_duplicates()
```

```
df.describe()
```

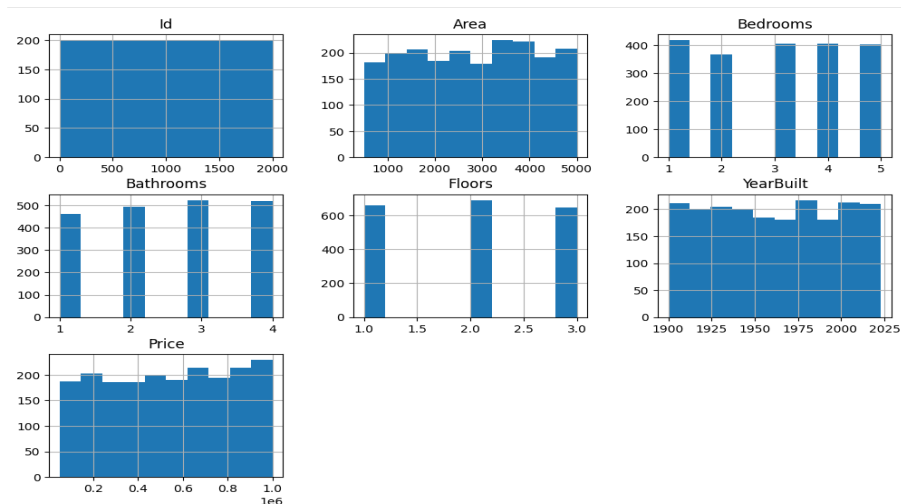
```
df.mean(numeric_only=True)
```

```
df.std(numeric_only=True)
```

```
Area                1295.146799
Bedrooms            1.424606
Bathrooms           1.108990
Floors              0.809188
YearBuilt           35.926695
Price              276428.845719
Location_Rural      0.425349
Location_Suburban   0.428100
Location_Urban      0.428702
Condition_Fair      0.439017
Condition_Good      0.421258
Condition_Poor      0.435123
Garage_Yes          0.499764
dtype: float64
```

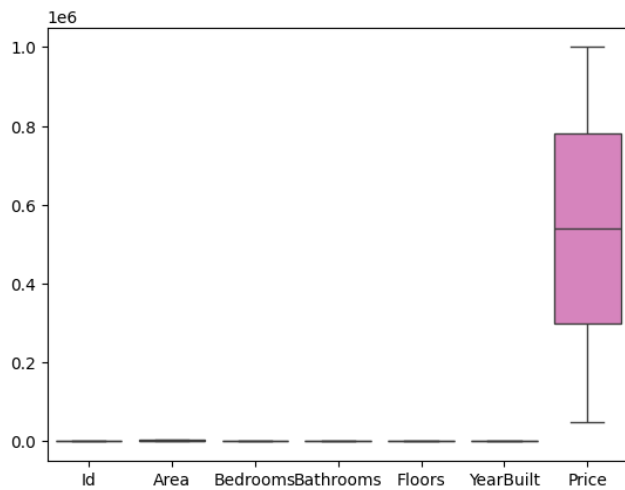
```
df.hist(figsize=(12,8))
```

```
plt.show()
```



```
sns.boxplot(data=df.select_dtypes(include=np.number))
```

```
plt.show()
```

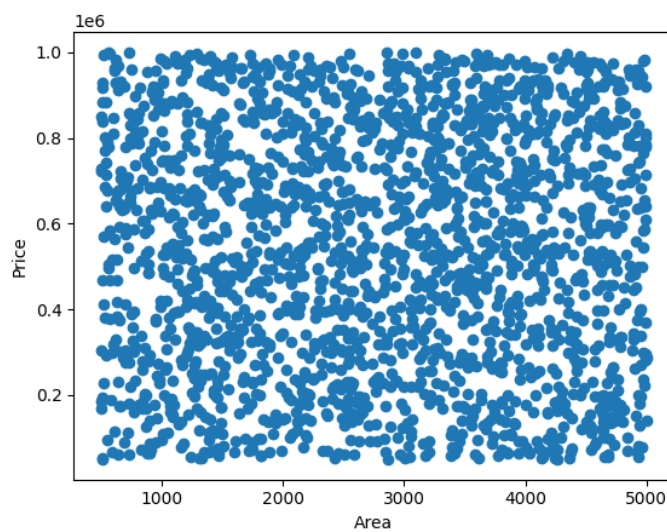


```
plt.scatter(df["Area"], df["Price"])
```

```
plt.xlabel("Area")
```

```
plt.ylabel("Price")
```

```
plt.show()
```



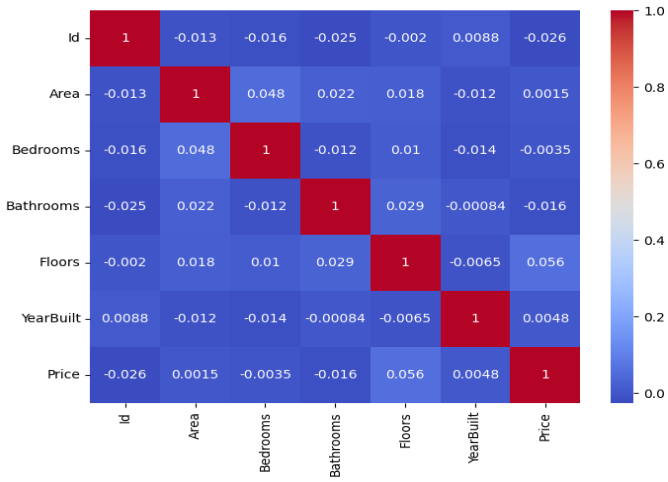
```
df.corr(numeric_only=True)
```

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Price
Id	1.000000	-0.012966	-0.016393	-0.025433	-0.002025	0.008840	-0.025643
Area	-0.012966	1.000000	0.047523	0.021881	0.017749	-0.011609	0.001542
Bedrooms	-0.016393	0.047523	1.000000	-0.011990	0.010435	-0.014125	-0.003471
Bathrooms	-0.025433	0.021881	-0.011990	1.000000	0.029089	-0.000839	-0.015737
Floors	-0.002025	0.017749	0.010435	0.029089	1.000000	-0.006474	0.055890
YearBuilt	0.008840	-0.011609	-0.014125	-0.000839	-0.006474	1.000000	0.004845
Price	-0.025643	0.001542	-0.003471	-0.015737	0.055890	0.004845	1.000000

```
plt.figure(figsize=(8,6))
```

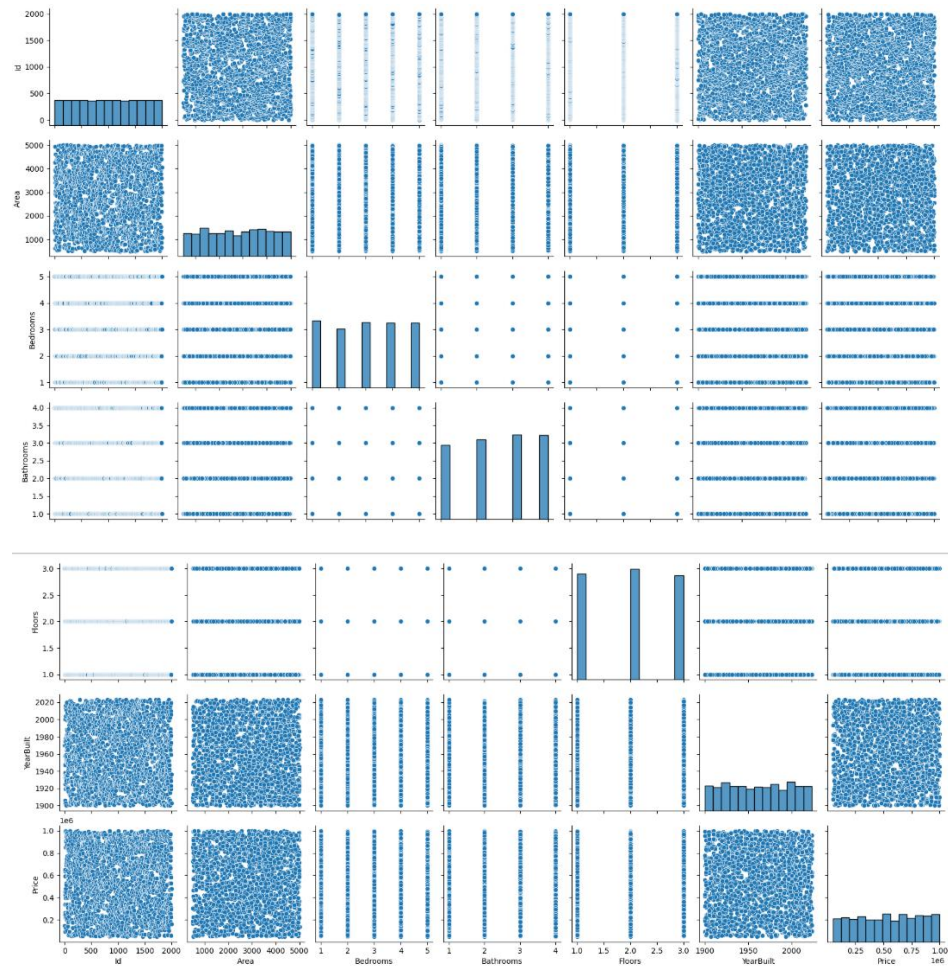
```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
```

```
plt.show()
```



```
sns.pairplot(df)
```

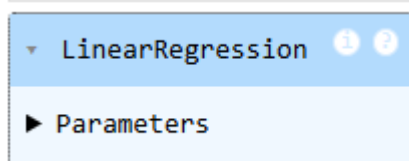
```
plt.show()
```



```

df.drop("Id", axis=1, inplace=True)
df = pd.get_dummies(df, drop_first=True)
X = df.drop("Price", axis=1)
y = df["Price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LinearRegression()
model.fit(X_train, y_train)

```



A screenshot of a Jupyter Notebook cell showing the `LinearRegression` class. The `Parameters` section is expanded, showing a table of parameters and their values.

LinearRegression		
Parameters		
	<code>fit_intercept</code>	<code>True</code>
	<code>copy_X</code>	<code>True</code>
	<code>tol</code>	<code>1e-06</code>
	<code>n_jobs</code>	<code>None</code>
	<code>positive</code>	<code>False</code>

```

y_pred = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))

```

```
print("R2 Score:", r2_score(y_test, y_pred))
```

MAE: 6154091.535304885

MSE: 38883270430729.23

RMSE: 6235645.149519754

R2 Score: -498.79249264381497

```
plt.scatter(y_test, y_pred)
```

```
plt.xlabel("Actual Price")
```

```
plt.ylabel("Predicted Price")
```

```
plt.show()
```

